



Servidor

Interactuar con el servidor opencode a través de HTTP.

El comando ``opencode serve`` ejecuta un servidor HTTP sin cabeza que expone un punto final OpenAPI que un cliente opencode puede usar.

Uso

```
opencode serve [--port <number>] [--hostname <string>] [--cors <origin>]
```

Opciones

BANDERA	DESCRIPCIÓN	PREDETERMINADO
<code>`--port`</code>	Puerto para escuchar	<code>`4096`</code>
<code>`--hostname`</code>	Nombre de host para escuchar	<code>`127.0.0.1`</code>
<code>`--mdns`</code>	Habilitar el descubrimiento de mDNS	<code>`false`</code>
<code>`--mdns-domain`</code>	Nombre de dominio personalizado para el servicio mDNS	<code>`opencode.local`</code>
<code>`--cors`</code>	Orígenes de navegador adicionales para permitir	<code>`[]`</code>

``--cors`` se puede pasar varias veces:

```
opencode serve --cors http://localhost:5173 --cors https://app.example.com
```

Autenticación

Configure ``OPENCODE_SERVER_PASSWORD`` para proteger el servidor con autenticación básica HTTP. El nombre de usuario predeterminado es ``opencode``, o configure ``OPENCODE_SERVER_USERNAME`` para anularlo. Esto se aplica tanto a ``opencode serve`` como a ``opencode web``.

```
OPENCODE_SERVER_PASSWORD=your-password opencode serve
```

Cómo funciona

Cuando ejecuta ``opencode``, inicia un TUI y un servidor. Donde el TUI es el Cliente que habla con el servidor. El servidor expone una especificación OpenAPI 3.1 punto final. Este punto final también se utiliza para generar un SDK.

CONSEJO

Utilice el servidor opencode para interactuar con opencode mediante programación.

Esta arquitectura permite que opencode admita múltiples clientes y le permite interactuar con opencode mediante programación.

Puede ejecutar ``opencode serve`` para iniciar un servidor independiente. Si tienes el opencode TUI ejecutándose, ``opencode serve`` iniciará un nuevo servidor.

Conectarse a un servidor existente

Cuando inicia el TUI, asigna aleatoriamente un puerto y un nombre de host. En su lugar, puede pasar ``--hostname`` y ``--port`` banderas. Luego use esto para

conectarse a su servidor.

El punto final `/tui` se puede utilizar para conducir el TUI a través del servidor. Por ejemplo, puede completar previamente o ejecutar un mensaje. Esta configuración es utilizada por los complementos OpenCode IDE.

Especificaciones

El servidor publica una especificación OpenAPI 3.1 que se puede ver en:

```
http://<hostname>:<port>/doc
```

Por ejemplo, `http://localhost:4096/doc`. Utilice la especificación para generar clientes o inspeccionar tipos de solicitudes y respuestas. O verlo en un explorador Swagger.

API

El servidor opencode expone las siguientes API.

Global

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>GET</code>	<code>/global/health</code>	Obtener el estado y la versión del servidor	<code>{ healthy: true, version: string }</code>
<code>GET</code>	<code>/global/event</code>	Obtenga eventos globales (transmisión SSE)	Flujo de eventos

Proyecto

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/project`</code>	Listar todos los proyectos	<code><u>`Project[]`</u></code>
<code>`GET`</code>	<code>`/project/current`</code>	Obtener el proyecto actual	<code><u>`Project`</u></code>

Ruta y VCS

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/path`</code>	Obtener la ruta actual	<code><u>`Path`</u></code>
<code>`GET`</code>	<code>`/vcs`</code>	Obtenga información de VCS para el proyecto actual	<code><u>`VcsInfo`</u></code>

Instancia

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`POST`</code>	<code>`/instance/dispose`</code>	Eliminar la instancia actual	<code>`boolean`</code>

Configuración

MÉTODO	ruta	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/config`</code>	Obtener información de configuración	<code><u>`Config`</u></code>
<code>`PATCH`</code>	<code>`/config`</code>	Actualizar configuración	<code><u>`Config`</u></code>
<code>`GET`</code>	<code>`/config/providers`</code>	Lista de proveedores y modelos predeterminados	<code><u>`{ providers: `Proveedor[]`, default: { [key: string]: string } }`</u></code>

Proveedor

MÉTODO	ruta	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/provider`</code>	Listar todos los proveedores	<code><u>`{ all: `Proveedor[]`, default: {...}, connected: string[] }`</u></code>
<code>`GET`</code>	<code>`/provider/auth`</code>	Obtener métodos de autenticación de proveedores	<code><u>`{ [providerID: string]: `ProviderAuthMethod[]` }`</u></code>
<code>`POST`</code>	<code>`/provider/{id}/oauth/authorize`</code>	Autorizar a un proveedor usando OAuth	<code><u>`ProviderAuthAuthorization`</u></code>
<code>`POST`</code>	<code>`/provider/{id}/oauth/callback`</code>	Manejar la devolución de llamada OAuth para un proveedor	<code><u>`boolean`</u></code>

Sesiones

MÉTODO	ruta	DESCRIPCIÓN	NOTAS
`GET`	`/session`	Listar todas las sesiones	Devuelve <u>`Session[]`</u>
`POST`	`/session`	Crear una nueva sesión	cuerpo: `{ parentID?, title? }`, devuelve <u>`Session`</u>
`GET`	`/session/status`	Obtener el estado de la sesión para todas las sesiones	Devuelve `{ [sessionID: string]: <u>`SessionStatus`</u> }`
`GET`	`/session/:id`	Obtener detalles de la sesión	Devuelve <u>`Session`</u>
`DELETE`	`/session/:id`	Eliminar una sesión y todos sus datos	Devuelve <u>`boolean`</u>
`PATCH`	`/session/:id`	Actualizar propiedades de sesión	cuerpo: `{ title? }`, devuelve <u>`Session`</u>
`GET`	`/session/:id/children`	Obtener las sesiones secundarias de una sesión	Devuelve <u>`Session[]`</u>
`GET`	`/session/:id/todo`	Obtener la lista de tareas pendientes para una sesión	Devuelve <u>`Todo[]`</u>
`POST`	`/session/:id/init`	Analizar aplicación y crear <u>`AGENTS.md`</u>	cuerpo: `{ messageID, providerID, modelID }`, devuelve <u>`boolean`</u>
`POST`	`/session/:id/fork`	Bifurca una sesión existente en un mensaje	cuerpo: `{ messageID? }`, devuelve <u>`Session`</u>
`POST`	`/session/:id/abort`	Cancelar una sesión en ejecución	Devuelve <u>`boolean`</u>

MÉTODO	ruta	DESCRIPCIÓN	NOTAS
<code>POST</code>	<code>/session/:id/share</code>	Compartir una sesión	Devuelve <code>Session</code>
<code>DELETE</code>	<code>/session/:id/share</code>	Dejar de compartir una sesión	Devuelve <code>Session</code>
<code>GET</code>	<code>/session/:id/diff</code>	Obtenga la diferencia para esta sesión	consulta: <code>messageID?</code> , devuelve <code>FileDiff[]</code>
<code>POST</code>	<code>/session/:id/summarize</code>	Resumir la sesión	cuerpo: <code>{ providerID, modelID }</code> , devuelve <code>boolean</code>
<code>POST</code>	<code>/session/:id/revert</code>	Revertir un mensaje	cuerpo: <code>{ messageID, partID? }</code> , devuelve <code>boolean</code>
<code>POST</code>	<code>/session/:id/unrevert</code>	Restaurar todos los mensajes revertidos	Devuelve <code>boolean</code>
<code>POST</code>	<code>/session/:id/permissions:permissionID</code>	Responder a una solicitud de permiso	cuerpo: <code>{ response, remember? }</code> , devuelve <code>boolean</code>

Mensajes

MÉTODO	ruta	DESCRIPCIÓN	NOTAS
<code>GET</code>	<code>/session/:id/message</code>	Listar mensajes en una sesión	consulta: <code>limit?</code> , devuelve <code>{ info: Mensaje, parts: Parte[] }</code>
<code>POST</code>	<code>/session/:id/message</code>	Envía un mensaje y espera respuesta	cuerpo: <code>{ messageID?, model?, agent?, noReply?, system?, tools?, parts }</code> , devuelve <code>{ info: Mensaje, parts: Parte[] }</code>

MÉTODO	RUTA	DESCRIPCIÓN	NOTAS
`GET`	`/session/:id/message/:messageID`	Obtener detalles del mensaje	Devuelve `{ info: <u>Mensaje</u> , parts: <u>Parte[]</u> }`
`POST`	`/session/:id/prompt_async`	Enviar un mensaje de forma asincrónica (sin espera)	cuerpo: igual que <code>/session/:id/message`</code> , devuelve <code>204 No Content`</code>
`POST`	`/session/:id/command`	Ejecutar un comando de barra diagonal	cuerpo: `{ messageId?, agent?, model?, command, arguments }`, devuelve `{ info: <u>Mensaje</u> , parts: <u>Parte[]</u> }`
`POST`	`/session/:id/shell`	Ejecute un comando de shell	cuerpo: `{ agent, model?, command }`, devuelve `{ info: <u>Mensaje</u> , parts: <u>Parte[]</u> }`

Comandos

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
`GET`	`/command`	Listar todos los comandos	<u>Command[]</u>

Archivos

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
`GET`	`/find?pattern=<pat>`	Buscar texto en archivos	Matriz de objetos coincidentes con <code>path`</code> , <code>lines`</code> , <code>line_number`</code> , <code>absolute_offset`</code> , <code>submatches`</code>
`GET`	`/find/file?query=	Buscar archivos y directorios por	<code>string[]`</code> (camino)

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
	<code><q></code>	nombre	
<code>`GET`</code>	<code>`/find/symbol?query=<q>`</code>	Buscar símbolos del espacio de trabajo	<u><code>`Symbol[]`</code></u>
<code>`GET`</code>	<code>`/file?path=<path>`</code>	Listar archivos y directorios	<u><code>`FileNode[]`</code></u>
<code>`GET`</code>	<code>`/file/content?path=<p>`</code>	Leer un archivo	<u><code>`FileContent`</code></u>
<code>`GET`</code>	<code>`/file/status`</code>	Obtener el estado de los archivos rastreados	<u><code>`File[]`</code></u>

Parámetros de consulta de `/find/file`

- ``query`` (obligatorio) – cadena de búsqueda (coincidencia aproximada)
- ``type`` (opcional): limita los resultados a ``"file"`` o ``"directory"``
- ``directory`` (opcional): anula la raíz del proyecto para la búsqueda.
- ``limit`` (opcional) – resultados máximos (1-200)
- ``dirs`` (opcional): indicador heredado (``"false"`` devuelve solo archivos)

Herramientas (experimentales)

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/experimental/tool/ids`</code>	Listar todos los ID de herramientas	<u><code>`ToolIDs`</code></u>

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/experimental/tool?provider=<p>&model=<m>`</code>	Listar herramientas con esquemas JSON para un modelo	<code>`ToolList`</code> <code>`-`</code>

LSP, formateadores y MCP

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/lsp`</code>	Obtener el estado del servidor LSP	<code>`LSPStatus[]`</code>
<code>`GET`</code>	<code>`/formatter`</code>	Obtener estado del formateador	<code>`FormatterStatus[]`</code>
<code>`GET`</code>	<code>`/mcp`</code>	Obtener el estado del servidor MCP	<code>`{ [name: string]: MCPStatus }`</code>
<code>`POST`</code>	<code>`/mcp`</code>	Agregue el servidor MCP dinámicamente	cuerpo: <code>`{ name, config }`</code> , devuelve MCP objeto de estado

Agentes

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/agent`</code>	Listar todos los agentes disponibles	<code>`Agent[]`</code>

Registro

MÉTODO	ruta	DESCRIPCIÓN	RESPUESTA
<code>POST</code>	<code>/log</code>	Escribir entrada de registro. Cuerpo: <code>{ service, level, message, extra? }</code>	<code>boolean</code>

TUI

MÉTODO	ruta	DESCRIPCIÓN	RESPUESTA
<code>POST</code>	<code>/tui/append-prompt</code>	Agregar texto al mensaje	<code>boolean</code>
<code>POST</code>	<code>/tui/open-help</code>	Abra el cuadro de diálogo de ayuda	<code>boolean</code>
<code>POST</code>	<code>/tui/open-sessions</code>	Abrir el selector de sesiones	<code>boolean</code>
<code>POST</code>	<code>/tui/open-themes</code>	Abra el selector de temas	<code>boolean</code>
<code>POST</code>	<code>/tui/open-models</code>	Abrir el selector de modelo	<code>boolean</code>
<code>POST</code>	<code>/tui/submit-prompt</code>	Enviar el mensaje actual	<code>boolean</code>
<code>POST</code>	<code>/tui/clear-prompt</code>	Borrar el mensaje	<code>boolean</code>
<code>POST</code>	<code>/tui/execute-command</code>	Ejecutar un comando (<code>{ command }</code>)	<code>boolean</code>
<code>POST</code>	<code>/tui/show-toast</code>	Mostrar brindis (<code>{ title?, message, variant }</code>)	<code>boolean</code>
<code>GET</code>	<code>/tui/control/next</code>	Espere la próxima solicitud de control	Objeto de solicitud de control

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`POST`</code>	<code>`/tui/control/response`</code>	Responder a una solicitud de control (<code>`{ body }`</code>)	<code>`boolean`</code>

Autenticación

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`PUT`</code>	<code>`/auth/:id`</code>	Establecer credenciales de autenticación. El cuerpo debe coincidir con el esquema del proveedor	<code>`boolean`</code>

Eventos

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/event`</code>	Transmisión de eventos enviados por el servidor. El primer evento es <code>`server.connected`</code> , luego eventos de bus	Transmisión de eventos enviados por el servidor

Documentación

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
<code>`GET`</code>	<code>`/doc`</code>	Especificación OpenAPI 3.1	Página HTML con especificación OpenAPI

 [Edita esta página](#)

© Anomaly

 [Found a bug? Open an issue](#)

Última actualización: 21 may 2026

 [Join our Discord community](#)

 [Español](#) 